DUNDALK INSTITUTE OF TECHNOLOGY
INSTITIÚID TEICNEOLAÍOCHTA DHÚN DEALGAN

Continuous Assessment
Cover Sheet

Student Name: Jamie McCourt		**Student Number:** D00265301
Programme: BSc (Hons) in Computing Systems and Operations (Software Development and DevOps)	**Stage:** 3	**Complete Student Checklist:** Re-read brief ■ References and Bibliography ■ Proofread ■
Module(s): Data Structures & Algorithms		
Due Date: 11/01/2026	**No. Pages:** 14	
Lecturer(s) Name: Michelle Graham		
Assignment No. and/or Description/Topic: Project analysing streaming platform user interactions using custom-built hash-based data structures (HashMap and HashSet) for deduplication, sorting, and user behaviour analysis with time/space complexity evaluation.		**Mode of Submission:** Softcopy ■ Hardcopy ☐

DECLARATION: **I declare that:**

- This work is entirely my own, and no part of it has been copied from any other person's words or ideas, except as specifically acknowledged through the use of inverted commas and in-text references;
- No part of this assignment has been written for me by any other person except where such collaboration has been authorised by the lecturer(s) concerned;
- I have not used generative artificial intelligence (AI) (e.g. ChatGPT) unless it has been permitted by the lecturer(s) concerned;
- I understand that I am bound by DkIT Academic Integrity Policy. I understand that I may be penalised if I have violated the policy in any way;
- This assignment has not been submitted for any other module at DkIT or any other institution, unless authorised by the relevant Lecturer(s);
- I have read and abided by all of the requirements set down for this assignment.

SIGNATURE…… **DATE**……

Lecturer's Comments:

Provisional Mark : _____ **Lecturers Signature :** _____ **Date:** _____

Streaming Platform Interaction Analytics: Project Overview and Key Findings

Student: Jamie McCourt (D00265301)
Project: Streaming Platform Analytics System
Module(s): Algorithms & Data Structures
Date: January 11, 2026

---

1. Introduction

This project implements a Java-based analytics system for processing user interaction data from a simulated streaming platform. My work focuses on two areas:

- Implementing custom hash-based data structures
- Designing algorithms for user behaviour analysis

I implemented `CustomHashMap` and `CustomHashSet` from first principles to demonstrate understanding of:

- Hash tables and collision resolution
- Load factor management and resizing
- Generic programming in Java

These structures support core functionality such as efficient filtering, deduplication, and user analytics.

The main problems addressed are:

1. Sorting interactions in reverse chronological order for display
2. Identifying users with the most diverse action types using hash-based aggregation
3. Deduplicating overlapping records across datasets using content-based hashing
4. Finding users with the highest interaction counts across datasets with timestamp-based tie-breaking

The report is structured as follows:

- Dataset description and design assumptions
- Algorithms for each major operation
- Custom data structure design and usage
- Complexity analysis and comparison with alternatives
- Design decisions and trade-offs

---

2. Data Description

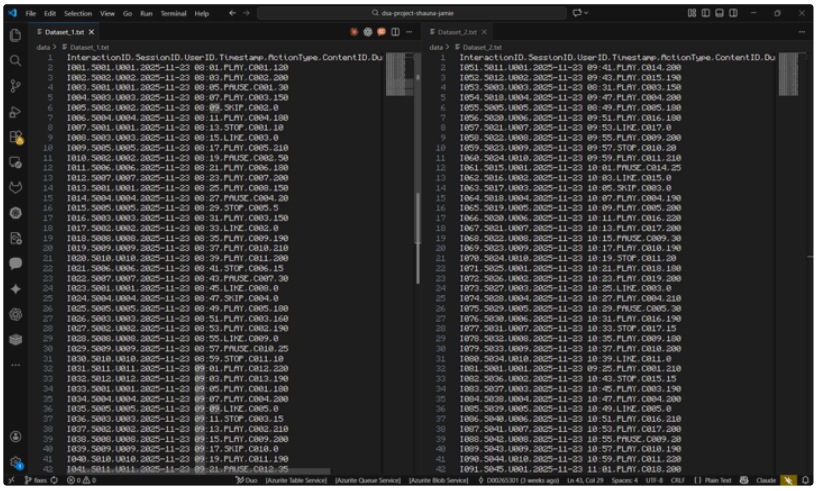


Figure 1 - Datasets used in the project

The dataset consists of `Interaction` objects, each with seven fields:

- `InteractionID`
- `SessionID`
- `UserID`
- `Timestamp` (`LocalDateTime`)
- `ContentID`
- `ActionType` (`PLAY`, `PAUSE`, `STOP`, `LIKE`, `SKIP`)
- `Duration` (seconds)

Two TXT files are used:

- `Dataset_1.txt`: 50 interactions
- `Dataset_2.txt`: 50 interactions

After deduplication, there are **97 unique interactions**, which is well below the <5,000-record upper bound that influenced my design.

Assumptions and design choices:

- `InteractionID` values are unique within each dataset.
- Two records are considered duplicates if all fields except `InteractionID` match (e.g., `I016` from dataset 1 equals `I053` from dataset 2).
- Timestamps are reliable and used as the primary ordering key.
- `Interaction.equals()` and `hashCode()` compare all fields except `InteractionID`, enabling **content-based deduplication**.

Given the small dataset (~100 records), I prioritised:

- Code clarity and learning value over micro-optimisation
- Demonstrating correct hash-table mechanics

- Using hash-based structures for O(1) average-case operations, even though simpler O(n) approaches would also be fast at this scale

---

3. Implemented Algorithms

3.1 Operation 1.2: Reverse Chronological Sorting

```java
/**
 * Gets all interactions sorted from newest to oldest (reverse chronological).
 * Converts the unordered CustomHashSet to a sorted List.
 *
 * @return List of interactions sorted by timestamp in descending order
 */
public List<Interaction> getInteractionsReverseChronological() {
    // Convert CustomHashSet to List
    List<Interaction> interactions = new ArrayList<>();
    for (Interaction interaction : uniqueInteractions) {
        interactions.add(interaction);
    }

    // Sort using the Interaction's built-in comparator (newest to oldest)
    Collections.sort(interactions, Interaction.reverseChronologicalOrder());

    return interactions;
}
```

```java
// ==================== COMPARATORS ====================

/**
 * Returns a Comparator for sorting interactions from newest to oldest.
 *
 * @return Comparator that sorts by timestamp in descending order
 */
public static Comparator<Interaction> reverseChronologicalOrder() {
    return Comparator.comparing(Interaction::getTimestamp).reversed();
}
```

```java
// ========================================
// OPERATION 1.2: Reverse Chronological Order
// ========================================
System.out.println("----------------------------------------");
System.out.println("1.2: Reverse Chronological Order");
System.out.println("----------------------------------------");

// Get interactions sorted newest to oldest
List<Interaction> reverseChronoInteractions = analyzer1.getInteractionsReverseChronological();
System.out.println("Showing " + reverseChronoInteractions.size() + " interactions (newest to oldest):");

// Display first 10 (most recent)
System.out.println("\nFirst 10 (newest):");
for (int i = 0; i < Math.min(10, reverseChronoInteractions.size()); i++) {
    Interaction inter = reverseChronoInteractions.get(i);
    System.out.printf("%s\t%s\t%s\t%s\t%s\t%d\n",
        inter.getInteractionId(), inter.getUserId(),
        inter.getTimestamp().format(FORMATTER),
        inter.getActionType(), inter.getContentId(),
        inter.getDurationSeconds());
}

System.out.println("...");
```

Figure 1 - Operation 1.2: Reverse Chronological Sorting

Goal:

Display all interactions from newest to oldest so that recent activity appears first.

Algorithm:

- Convert the internal `CustomHashSet<Interaction>` (unordered) to an `ArrayList<Interaction>`.
- Use `Collections.sort()` with a custom `Comparator`.
- Define a static comparator in `Interaction`:

This uses Java's built-in **TimSort**.

Reasoning:

- TimSort provides O(n log n) worst-case performance and is highly optimised for real-world data. (At the time, slight difficulty with creating an algorithm so having less time I opted for Java's TimSort)
- It is stable and production-tested.
- Re-implementing merge sort or quicksort would add significant code without any practical benefit for ~100 records.

3.2 Operation 1.3: User with Most Diverse Action Types

```java
public String getUserWithMostDiverseActions() {
    if (userIndex.isEmpty()) {
        return null;
    }

    // Step 1: Get diversity counts for all users
    Map<String, Integer> diversityCounts = getUserActionTypeCounts();

    // Step 2: Find the maximum diversity count
    int maxDiversity = 0;
    for (int diversityCount : diversityCounts.values()) {
        if (diversityCount > maxDiversity) {
            maxDiversity = diversityCount;
        }
    }

    // Step 3: Collect all users with the maximum diversity (potential ties)
    List<String> tiedUsers = new ArrayList<>();
    for (Map.Entry<String, Integer> entry : diversityCounts.entrySet()) {
        if (entry.getValue() == maxDiversity) {
            tiedUsers.add(entry.getKey());
        }
    }

    // Step 4: If only one user has max diversity, return immediately
    if (tiedUsers.size() == 1) {
        return tiedUsers.get(0);
    }

    // Step 5: Multiple users tied - apply earliest timestamp tie-breaker
    String winnerUser = tiedUsers.get(0);
    LocalDateTime winnerTimestamp = getUserEarliestTimestamp(winnerUser);

    for (int i = 1; i < tiedUsers.size(); i++) {
        String currentUser = tiedUsers.get(i);
        LocalDateTime currentTimestamp = getUserEarliestTimestamp(currentUser);

        // User with earlier timestamp wins (pioneered diverse behavior first)
        if (currentTimestamp != null &&
            (winnerTimestamp == null || currentTimestamp.isBefore(winnerTimestamp))) {
            winnerUser = currentUser;
            winnerTimestamp = currentTimestamp;
        }
    }

    // Step 6: Return the winner
    return winnerUser;
```

```java
 * @see #getUserWithMostDiverseActions() - Main method using this helper
 * @see #getUserEarliestTimestamp(String) - Tie-breaker for diversity ties
 */
public int countUniqueActions(String userId) {
    List<Interaction> userInteractions = getInteractionsByUser(userId);

    if (userInteractions.isEmpty()) {
        return 0;
    }

    // Use a HashSet to track unique action types
    Set<String> uniqueActions = new HashSet<>();
    for (Interaction interaction : userInteractions) {
        uniqueActions.add(interaction.getActionType());
    }

    return uniqueActions.size();
}
```

```java
     * @see #countUniqueActions(String) - Helper method calculating per-user diversity
     * @see #getUserWithMostDiverseActions() - Main method using this aggregation
     * @see #getUserEarliestTimestamp(String) - Tie-breaker for diversity ties
     */
    public Map<String, Integer> getUserActionTypeCounts() {
        Map<String, Integer> diversityCounts = new HashMap<>();

        // Iterate through all users and calculate their diversity
        for (String userId : userIndex.keySet()) {
            int diversityCount = countUniqueActions(userId);
            diversityCounts.put(userId, diversityCount);
        }

        return diversityCounts;
    }
```

```java
// ========================================
// OPERATION 1.3: Most diverse user
// ========================================
System.out.println("----------------------------------------");
System.out.println("1.3: User with Most Different Actions");
System.out.println("----------------------------------------");

String mostDiverseUser = analyzer1.getUserWithMostDiverseActions();
Map<String, Integer> diversityCounts = analyzer1.getUserActionTypeCounts();

if (mostDiverseUser != null) {
    int maxDiversity = diversityCounts.get(mostDiverseUser);
    LocalDateTime earliestTimestamp = analyzer1.getUserEarliestTimestamp(mostDiverseUser);

    System.out.println("Winner: " + mostDiverseUser);
    System.out.println("Unique action types: " + maxDiversity);
    System.out.println("Earliest interaction: " + earliestTimestamp.format(FORMATTER));

    // Show tied users (if any)
    System.out.println("\nTied users (all have " + maxDiversity + " unique actions):");
    for (Map.Entry<String, Integer> entry : diversityCounts.entrySet()) {
        if (entry.getValue() == maxDiversity) {
            LocalDateTime timestamp = analyzer1.getUserEarliestTimestamp(entry.getKey());
            String tiebreaker = entry.getKey().equals(mostDiverseUser) ? " ← WINNER (tie-breaker)" : "";
            System.out.println("  " + entry.getKey() + ": earliest at " +
                               timestamp.format(FORMATTER) + tiebreaker);
        }
    }

    // Show other users for comparison
    System.out.println("\nOther users:");
    for (Map.Entry<String, Integer> entry : diversityCounts.entrySet()) {
        if (entry.getValue() < maxDiversity) {
            System.out.println("  " + entry.getKey() + ": " + entry.getValue() + " unique actions");
        }
    }
} else {
    System.out.println("No users found in dataset.");
}
System.out.println();
```

Figure 2 - Operation 1.3: User with Most Diverse Action Types

Goal:

Identify the user in `Dataset_1` who performs the widest variety of actions (`PLAY` , `PAUSE` , `STOP` , `LIKE` , `SKIP`). This measures **behavioural diversity** (variety of actions), not volume.

Algorithm:

`getUserWithMostDiverseActions()` :

1. **Aggregate diversity per user**
   - Iterate through `userIndex` (`UserID → List<Interaction>`).
   - For each user, call `countUniqueActions(userId)` :
     - Build a `HashSet<String>` of action types from their interactions.
     - Diversity count = `uniqueActions.size()` .
2. **Find maximum diversity**
   - Scan the diversity map to find the highest count.
3. **Collect ties**
   - Build a list of users with the maximum diversity.
4. **Early return**
   - If there is only one user, return immediately.
5. **Tie-breaking** (earliest timestamp wins)

- For tied users, call `getUserEarliestTimestamp(userId)` and select the user with the earliest interaction.

6. **Return winner** (`userId`).

Complexity:

- Aggregation: O(n × m), where n = number of users, m = average interactions per user (≈ 5).
- Tie-breaking: O(k × m), where k = number of tied users (typically small).

Design choice:

- Using `HashSet` gives O(1) average-case insertion and deduplication of action types.
- This maps directly to “unique actions per user”.

Alternative considered (rejected):

- **Bitmask representation** (one bit per action type).
  - Saves a few bytes per user but harms readability and introduces encoding/decoding logic.
  - Memory savings are negligible for ~100 records; clarity is more important in my opinion.

3.3 Operation 2.2: Deduplicating Across Datasets

```
 * @param dataset1 First list of interactions (e.g., Dataset_1.txt)
 * @param dataset2 Second list of interactions (e.g., Dataset_2.txt)
 * @return List of unique interactions with duplicates removed (97 for provided datasets)
 * @see org.example.model.Interaction#equals(Object)
 * @see org.example.model.Interaction#hashCode()
 */
public static List<Interaction> deduplicateDatasets(List<Interaction> dataset1, List<Interaction> dataset2) {
    // Create CustomHashSet with appropriate capacity
    int initialCapacity = Math.max(16, dataset1.size() + dataset2.size());
    CustomHashSet<Interaction> uniqueSet = new CustomHashSet<>(initialCapacity);

    // Add all interactions from both datasets
    // CustomHashSet automatically removes duplicates during add()
    for (Interaction interaction : dataset1) {
        uniqueSet.add(interaction);
    }
    for (Interaction interaction : dataset2) {
        uniqueSet.add(interaction);
    }

    // Convert CustomHashSet to List for return
    List<Interaction> result = new ArrayList<>();
    for (Interaction interaction : uniqueSet) {
        result.add(interaction);
    }

    return result;
}
```

```
/**
 * Checks if two interactions are "duplicates".
 * We ignore InteractionID because different datasets might have different IDs
 * but the same actual interaction data.
 *
 * Like saying: "These two records describe the same event, even if they
 * have different labels."
 */
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;

    Interaction other = (Interaction) obj;

    // Two interactions are duplicates if ALL fields except InteractionID match
    return Objects.equals(sessionId, other.sessionId) &&
           Objects.equals(userId, other.userId) &&
           Objects.equals(timestamp, other.timestamp) &&
           Objects.equals(actionType, other.actionType) &&
           Objects.equals(contentId, other.contentId) &&
           durationSeconds == other.durationSeconds;
}
```

```java
/**
 * Hash code for efficient lookup in HashSet/HashMap.
 * Think of this as a fingerprint for the interaction.
 */
@Override
public int hashCode() {
    return Objects.hash(sessionId, userId, timestamp, actionType, contentId, durationSeconds);
}
```

```java
// ========================================
// OPERATION 2.2: Remove Duplicates
// ========================================
System.out.println("----------------------------------------");
System.out.println("2.2: Deduplicate Datasets");
System.out.println("----------------------------------------");

// Deduplicate datasets - remove interactions from dataset2 that appear in dataset1
List<Interaction> deduplicatedData = InteractionAnalyzer.deduplicateDatasets(dataset1, dataset2);

System.out.println("Dataset 1: " + dataset1.size() + " interactions");
System.out.println("Dataset 2: " + dataset2.size() + " interactions");
System.out.println("Total: " + (dataset1.size() + dataset2.size()) + " interactions");
System.out.println("After deduplication: " + deduplicatedData.size() + " unique interactions");
System.out.println("Duplicates removed: " + (dataset1.size() + dataset2.size() - deduplicatedData.size()));
System.out.println();
```

Goal:

Combine `Dataset_1` and `Dataset_2` and remove duplicates where all fields match except `InteractionID`.

Result: 100 → **97 unique interactions** (3 duplicates removed).

Algorithm:

`deduplicateDatasets(dataset1, dataset2)`:

1. Create `CustomHashSet<Interaction> seen`.
2. Create `ArrayList<Interaction> result`.
3. **Phase 1 – dataset 1:**
   - For each interaction in `dataset1`:
     - `seen.add(interaction)`
     - `result.add(interaction)`
4. **Phase 2 – dataset 2:**
   - For each interaction in `dataset2`:
     - If `!seen.contains(interaction)`, add it to both `seen` and `result`.

The key is that `Interaction.equals()` and `hashCode()` ignore `InteractionID` and compare content fields only.

Complexity:

- Expected O(n) time, where n = total interactions ($n_1 + n_2$).
- O(n) space for the hash set.

Alternative considered (rejected):

- **Sorting + adjacent duplicate scan**
  - Requires O(n log n) time for sorting by a composite key.
  - Needs a complex comparator combining six fields.
  - Does not give any benefit since sorted order is not required.
- **Nested loops (brute-force)**
  - $O(n^2)$ comparisons, clearly poor practice even at this scale.

3.4 Operation 2.3: User with Most Interactions

```java
public String getUserWithMostInteractionsAcrossDatasets() {
    if (userIndex.isEmpty()) {
        return null;
    }

    // Step 1: Get all user interaction counts
    Map<String, Integer> counts = getUserInteractionCounts();

    // Step 2: Find the maximum interaction count
    int maxCount = 0;
    for (int count : counts.values()) {
        if (count > maxCount) {
            maxCount = count;
        }
    }

    // Step 3: Collect all users with the maximum count (potential ties)
    List<String> tiedUsers = new ArrayList<>();
    for (Map.Entry<String, Integer> entry : counts.entrySet()) {
        if (entry.getValue() == maxCount) {
            tiedUsers.add(entry.getKey());
        }
    }

    // Step 4: If only one user has max count, return immediately
    if (tiedUsers.size() == 1) {
        return tiedUsers.get(0);
    }

    // Step 5: Multiple users tied - apply timestamp tie-breaker
    String winnerUser = tiedUsers.get(0);
    LocalDateTime winnerTimestamp = getUserMostRecentTimestamp(winnerUser);

    for (int i = 1; i < tiedUsers.size(); i++) {
        String currentUser = tiedUsers.get(i);
        LocalDateTime currentTimestamp = getUserMostRecentTimestamp(currentUser);

        // User with more recent timestamp wins
        if (currentTimestamp != null &&
            (winnerTimestamp == null || currentTimestamp.isAfter(winnerTimestamp))) {
            winnerUser = currentUser;
            winnerTimestamp = currentTimestamp;
        }
    }

    return winnerUser;
}
```

```java
/**
 * Aggregates interaction counts for all users in the dataset.
 * Returns a map where each key is a userId and the value is the count
 * of unique interactions for that user (after deduplication).
 *
 * This method is used for identifying the most active users and
 * handling tie-breaking scenarios in multi-dataset analysis.
 *
 * Example output for deduplicated datasets:
 * {U001=12, U002=12, U003=12, U004=12, U005=12, U006=10, U007=10, ...}
 *
 * @return Map of userId to interaction count, or empty map if no data
 */
public Map<String, Integer> getUserInteractionCounts() {
    Map<String, Integer> counts = new HashMap<>();

    // Iterate through userIndex and extract sizes
    for (Map.Entry<String, List<Interaction>> entry : userIndex.entrySet()) {
        String userId = entry.getKey();
        int count = entry.getValue().size();
        counts.put(userId, count);
    }

    return counts;
}
```

```java
/**
 * Gets the timestamp of the most recent interaction for a specific user.
 * Used for tie-breaking when multiple users have the same interaction count.
 *
 * The user with the most recent (latest) timestamp is considered more active
 * in tie-breaking scenarios.
 *
 * Example use case:
 * - U001 has 12 interactions, most recent at 11:01
 * - U005 has 12 interactions, most recent at 11:09
 * - U005 wins the tie-breaker (11:09 > 11:01)
 *
 * @param userId The user ID to check
 * @return The most recent timestamp for this user, or null if user not found
 */
public LocalDateTime getUserMostRecentTimestamp(String userId) {
    List<Interaction> userInteractions = getInteractionsByUser(userId);

    if (userInteractions.isEmpty()) {
        return null;
    }

    LocalDateTime mostRecent = userInteractions.get(0).getTimestamp();

    for (Interaction interaction : userInteractions) {
        LocalDateTime timestamp = interaction.getTimestamp();
        if (timestamp.isAfter(mostRecent)) {
            mostRecent = timestamp;
        }
    }

    return mostRecent;
}
```

```java
// ========================================
// OPERATION 2.3: User with Most Interactions (Multi-Dataset)
// ========================================
System.out.println("----------------------------------------");
System.out.println("2.3: User with Most Interactions (After Deduplication)");
System.out.println("----------------------------------------");

// Create analyzer with deduplicated data
InteractionAnalyzer multiDatasetAnalyzer = new InteractionAnalyzer(deduplicatedData);

String mostActiveUser = multiDatasetAnalyzer.getUserWithMostInteractionsAcrossDatasets();
Map<String, Integer> interactionCounts = multiDatasetAnalyzer.getUserInteractionCounts();

if (mostActiveUser != null) {
    int maxCount = interactionCounts.get(mostActiveUser);
    LocalDateTime latestTimestamp = multiDatasetAnalyzer.getUserMostRecentTimestamp(mostActiveUser);

    System.out.println("Winner: " + mostActiveUser);
    System.out.println("Total interactions: " + maxCount);
    System.out.println("Most recent interaction: " + latestTimestamp.format(FORMATTER));

    // Show tied users (if any)
    System.out.println("\nTied users (all have " + maxCount + " interactions):");
    for (Map.Entry<String, Integer> entry : interactionCounts.entrySet()) {
        if (entry.getValue() == maxCount) {
            LocalDateTime timestamp = multiDatasetAnalyzer.getUserMostRecentTimestamp(entry.getKey());
            String tiebreaker = entry.getKey().equals(mostActiveUser) ? " <- WINNER (tie-breaker)" : "";
            System.out.println("  " + entry.getKey() + ": most recent at " +
                               timestamp.format(FORMATTER) + tiebreaker);
        }
    }

    // Show other users for comparison
    System.out.println("\nOther users:");
    for (Map.Entry<String, Integer> entry : interactionCounts.entrySet()) {
        if (entry.getValue() < maxCount) {
            System.out.println("  " + entry.getKey() + ": " + entry.getValue() + " interactions");
        }
    }
} else {
    System.out.println("No users found in deduplicated dataset.");
}
System.out.println();
```

Figure 3 - Operation 2.3: User with Most Interactions

Goal

After deduplication, find the user with the **highest number of interactions** across both datasets.

Ties are broken by **most recent timestamp**, rewarding sustained, recent activity.

Algorithm:

`getUserWithMostInteractionsAcrossDatasets()`:

1. **Aggregate interaction counts**
   - Iterate through `userIndex` and store `interactionList.size()` per `userId` in a map.
2. **Find maximum count**

- Scan the map to find the highest count.

3. **Collect ties**
   - Build a list of users with this maximum count.
4. **Early return** if there is a single winner.
5. **Tie-breaking** (latest timestamp wins)
   - Use `getUserMostRecentTimestamp(userId)` to find each tied user's latest interaction.
   - Select the user with the most recent timestamp.

Complexity:

- Aggregation: O(n) over users.
- Tie-breaking: O(k × m), where k = tied users, m = average interactions.

Design choice:

- The **latest** timestamp is appropriate for "most active" users, in contrast with the **earliest** timestamp used for diversity.
- This deliberate asymmetry matches the meaning of each metric.

---

4. Data Structures Used

4.1 CustomHashMap<K, V>

```java
package org.example.structure;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

/**
 * Custom implementation of a HashMap using collision chaining.
 * Generic key-value storage with automatic resizing at 75% load factor.
 * Uses ArrayList for bucket chaining to handle collisions.
 *
 * @param <K> Key type
 * @param <V> Value type
 */
public class CustomHashMap<K, V> {

    /**
     * Inner class representing a key-value pair entry.
     */
    private static class Entry<K, V> {
        final K key;
        V value;

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }

        @Override
        public String toString() {
            return key + "=" + value;
        }
    }

    // Array of buckets (each bucket is an ArrayList for chaining)
    private ArrayList<Entry<K, V>>[] buckets;

    // Number of key-value pairs stored
    private int size;
```

Figure 4 - Code from HashMap

Overview:

A generic hash map with:

- Average-case O(1) for `put`, `get`, `remove`, `containsKey`
- Separate chaining for collision handling
- Automatic resizing based on a 0.75 load factor

Key design elements:

- **Internal structure**
  - Array of buckets (initial capacity 16, doubled on resize).
  - Each bucket is a `LinkedList` of `Entry<K, V>` nodes.
- **Hashing and indexing**
- **Collision handling**
  - Linear traversal of the bucket chain, using `equals()` to match keys.
  - Updates existing entries or appends new ones.
- **Load factor and resizing**
  - When `size > capacity × 0.75`, the table is doubled and all entries rehashed.

Usage in project:

- `userIndex`: `Map<String, List<Interaction>>`
- `actionIndex`: `Map<String, List<Interaction>>`
- Count maps for interactions and action diversity

4.2 CustomHashSet<T>

```
* @param <E> The type of elements maintained by this set
*
* @author Jamie McCourt (D00265301)
* @version 1.0
*/
ublic class CustomHashSet<E> implements Iterable<E> {

   // Internal storage - uses CustomHashMap with dummy values
   private final CustomHashMap<E, Object> map;

   // Dummy value to associate with keys in the backing map
   // All keys map to this single object to minimize memory usage
   private static final Object PRESENT = new Object();

   /**
    * Constructs an empty CustomHashSet with default initial capacity (16).
    * The set will automatically resize when load factor exceeds 75%.
    */
   public CustomHashSet() {
       this.map = new CustomHashMap<>();
   }

   /**
    * Constructs an empty CustomHashSet with specified initial capacity.
    * Use this constructor when you know the approximate size in advance
    * to minimize rehashing operations.
    *
    * @param initialCapacity The initial capacity of the hash set
    * @throws IllegalArgumentException if initialCapacity is non-positive
    */
   public CustomHashSet(int initialCapacity) {
       this.map = new CustomHashMap<>(initialCapacity);
   }
```

Figure 5 - Code from HashSet

Overview:

A generic set built on top of `CustomHashMap<T, Boolean>` to enforce uniqueness.

Core operations:

- `add(T element)` → delegates to `map.put(element, TRUE)`
- `contains(T element)` → `map.containsKey(element)`
- `remove(T element)` → `map.remove(element)`
- `size()` → `map.size()`

Usage in project:

- `uniqueInteractions`: deduplication across datasets
- Tracking seen interactions in `deduplicateDatasets()`
- Internal use for per-user action diversity where appropriate

---

4.3 Integration with Standard Collections

I deliberately use standard Java collections where custom implementation would not be of any value (Or if I encountered time-restraints):

- **ArrayList**:
  - Dynamic arrays and integration with `Collections.sort()` for ordered views.
- **LinkedList**:
  - Used for collision chains in `CustomHashMap`.
- **HashSet**:
  - Used inside `countUniqueActions()` for per-user action type deduplication.
- **Interfaces (** `List`, `Set`, `Map`):
  - Used in method signatures for flexibility.

This hybrid approach shows that I can both implement data structures and choose when the standard library is the appropriate, professional choice.

---

5. Complexity Analysis

5.1 Reverse Chronological Sorting

- **Time:** O(n log n) using TimSort
- **Space:** O(n) extra
- **Alternatives:** Custom merge sort or quicksort have the same asymptotic behaviour and no practical advantage for ~100 elements.

5.2 Hash-Based Indexing (userIndex, actionIndex)

- **Build time:** O(n) over all interactions
- **Query time:** O(1) average for lookups
- **Space:** O(n) for maps and lists

Without indexing, each query would require an O(n) scan. The indexing cost is justified even at modest scales, and aligns with best practice in analytics systems.

5.3 Diversity Analysis

- **Time:** O(n × m + k × m)
- **Space:** O(n) for diversity counts and tie lists
- **Alternative (bitmask):** Same time complexity, slightly less memory, but lower readability and maintainability.

5.4 Deduplication

- **Time:** Expected O(n) using `CustomHashSet`
- **Space:** O(n) for the set and result list
- **Alternatives:**
  - Sorting-based: O(n log n) + comparator complexity
  - Nested loops: $O(n^2)$, unacceptable design even at small sizes (Best to avoid at all costs)

5.5 Most Interactions

- **Time:** O(n + k × m)
- **Space:** O(n) for count maps and tie lists
- **Alternative:** Maintain live counters during ingestion.
  - Would make queries O(1) but complicate construction logic and is unnecessary for batch analysis.

---

6. Design Decisions and Trade-offs

- **Custom vs standard structures:**
  - Custom for `HashMap` / `HashSet` to demonstrate understanding & allowed for easier deduplication and searching.
  - Standard `ArrayList`, `LinkedList`, `HashSet` to keep code focused and idiomatic.
- **Indexing:**
  - Chosen to favour query speed and realistic analytics behaviour.
- **Batch processing:**
  - Computations are performed on-demand over indexed data rather than maintained incrementally.
- **Tie-breaking strategies:**
  - Earliest timestamp for diversity (who became diverse first).
  - Latest timestamp for activity (who is currently most active).
- **Generics and load factor:**
  - Full generic implementations (`<K, V>`, `<T>`) and a 0.75 load factor mirror industry practice and Java's own collections.

---

7. Appendix (Selected Pseudocode)

7.1 Deduplication Using CustomHashSet

```
FUNCTION deduplicateDatasets(dataset1, dataset2):
    // Initialize hash set for tracking seen interactions
    seen ← new CustomHashSet<Interaction>()
    result ← new ArrayList<Interaction>()

    // Phase 1: Add all interactions from dataset1
    // (No duplicates exist within a single dataset by assumption)
    FOR EACH interaction IN dataset1:
        seen.add(interaction)          // O(1) average insertion
        result.add(interaction)
    END FOR

    // Phase 2: Add only non-duplicate interactions from dataset2
    FOR EACH interaction IN dataset2:
        IF NOT seen.contains(interaction):   // O(1) average lookup
            seen.add(interaction)
            result.add(interaction)
        END IF
    END FOR

    RETURN result    // 97 unique from 100 total
END FUNCTION

// Note: Uses Interaction.equals() and hashCode() which compare
// all fields EXCEPT InteractionID to detect content duplicates
```

Uses `Interaction.equals()` / `hashCode()` that ignore `InteractionID`.

---

8. Testing Summary

- **Deduplication:**
  - Input: 50 + 50 interactions
  - Output: 97 unique interactions
  - Duplicates correctly identified and removed.
- **Diversity (Operation 1.3):**
  - Winner: `U001` with 4 unique actions, earliest at 08:01.
- **Most interactions (Operation 2.3):**
  - Winner: `U002` with 12 interactions, latest at 11:03.

---

9. Key Learnings

- Load factor management is essential for maintaining O(1) hash table performance.
- Generic implementation improves type safety and reusability but requires understanding type erasure.
- Tie-breaking strategies must reflect the semantics of each metric.
- Custom data structures show conceptual understanding; appropriate use of the Java standard library shows professional judgment.
- Hash-based indexing significantly improves query performance even on small datasets.
- Complexity analysis must balance asymptotic guarantees with real dataset constraints.